

ARAMIREZ-AI

Diseño Técnico

Sistema de Notificaciones — Arquitectura Propuesta

Organización	Gerencia de Desarrollo y Aplicaciones
Preparado por	Software Architect
Fecha	28 Jun 2026
Clasificación	Interno

1. Contexto y Requerimientos

Se requiere un sistema de notificaciones escalable capaz de procesar 10,000 mensajes por día a través de tres canales: email, push y SMS. El sistema debe soportar plantillas dinámicas basadas en Handlebars, reintentos con backoff exponencial y latencia extrema a extrema menor a 5 segundos.

- Requerimiento funcional: envío multicanal con plantillas dinámicas (email, push, SMS)
- Requerimiento no funcional: latencia < 5s en el percentil 99
- Requerimiento no funcional: throughput sostenido de 500 msg/min
- Requerimiento no funcional: disponibilidad 99.9% en horario laboral
- Requerimiento de negocio: tracking de apertura y entrega por canal

Criterio	Opción A: Síncrona	Opción B: Basada en Colas (Recomendada)
Latencia P99	< 100ms	< 1s
Throughput máximo	500 msg/s	10,000 msg/s
Costo incremental de infraestructura	\$0 (misma instancia)	\$200/mes (cluster RabbitMQ)
Complejidad operativa	Baja	Media
Resiliencia ante fallos	Baja (pérdida de mensajes)	Alta (colas durables + DLQ)
Escalabilidad horizontal	Limitada	Alta (particiones por canal)

2. Diseño Detallado

Componente	Responsabilidad	Tecnología Propuesta	Cantidad
API Receiver	Recibe solicitudes y las publica en la cola	Node.js + Express	2 réplicas
RabbitMQ Cluster	Buffer duradero con colas por canal	RabbitMQ 3.13	3 nodos
Email Worker	Procesa cola de email (SMTP)	Node.js + Nodemailer	2 workers
Push Worker	Procesa cola de push (FCM/APNs)	Node.js + firebase-admin	2 workers
SMS Worker	Procesa cola de SMS (Twilio)	Node.js + Twilio SDK	1 worker
Dead Letter Queue	Almacena mensajes fallidos para reproceso	RabbitMQ DLQ	—

Trade-off Identificado

La Opción B introduce latencia adicional (~1s) debido al encolamiento, pero ofrece escalabilidad horizontal, resiliencia ante fallos y capacidad de crecimiento sin cambios arquitectónicos. Para el volumen actual (10k msg/día) la latencia es aceptable y cumple con el SLA de 5 segundos.

Opción B — Arquitectura Basada en Colas

Problema: La Opción A (síncrona) no escala al volumen proyectado a 12 meses (50,000 msg/día) y no ofrece resiliencia ante fallos del proveedor de email o push

Recomendación: Implementar arquitectura basada en colas con RabbitMQ, workers dedicados por canal y dead letter queue para manejo de fallos

Acciones sugeridas:

- Provisionar cluster RabbitMQ de 3 nodos en Kubernetes
- Implementar worker pool de 4 consumidores por canal
- Configurar dead letter queue con TTL de 7 días
- Implementar backoff exponencial: 1s, 5s, 30s, 5min, 30min
- Agregar paneles de monitoreo en Grafana para profundidad de colas y tasa de fallos



Se estima que la implementación completa tomará 3 sprints (4 semanas). Próximo paso: presentar al comité de arquitectura para aprobación y definición de fechas de inicio.